

MACHINE LEARNING IN FINANCE

MODULE 6 | LESSON 1

INTRODUCTION TO DEEP LEARNING

Reading Time 70 minutes

Prior Knowledge Machine learning

Keywords Deep learning, activation function, forward propagation, backpropagation

In this lesson, we are going to give a theoretical background of neural networks that will give you the intuition needed to master deep learning beyond the module. Once you are done with this lesson, you should be able to understand where deep learning is applicable, why it is needed, and the building blocks of neural networks. To ensure the accuracy provided by the neural networks is high, the concept of backpropagation will be introduced. The lesson will set the momentum for upcoming lessons that will heavily rely on this lesson.

1. Deep Learning

Deep learning is a field of machine learning that has been inspired by the functionality of the human brain neural networks (Zhang et al). In recent years, deep learning has attracted immense use cases due to its strong generalization capabilities and robust training power. When the data is huge, it has been shown that deep learning performs much better than classical machine learning algorithms.

Deep learning has successfully been applied in both classification and regression problems in computer visions, natural language processing, audio-visual recognition, and image processing. In the recent past, deep learning applications have been researched and implemented in the fields of finance, medicine, operations research, and banking. The applications of deep learning in finance have been fueled by the existence of vast financial data, which happens to be incomplete, uncertain, and inconsistent, thereby posing a huge challenge to the traditional statistical methods that were previously employed to perform financial data analysis.

Applications of deep learning in the financial world include:

- Credit risk modeling (Addo et al. 38; Modarres et al.)
- Forecasting of option prices, future prices, and share prices (Sezer et al.; Yao et al. 455-466)
- Predicting bankruptcy (Mai et al. 743-758; Qu et al. 895-899)

The advancement of deep learning and machine learning in the areas of finance has largely been facilitated by:

- Evolution of computing power.
- Advancement of mathematical theory by machine learning researchers.
- Accessibility of technology.
- Breakthrough achieved in other domains of deep learning such as in games, natural language processing, image processing, etc.

In financial engineering, the following problems can be solved using deep learning algorithms:

- Deep hedging (Buehler et al. 1271-1291)
- Deep calibration (Nixon et al.)
- Deep simulation. (Maeda et al. 71)

Now that we have introduced deep learning and its application areas, in this lesson, we will learn about the following topics:

1. Deep learning
2. Forward propagation
3. Backward propagation

Deep learning derives its name from artificial neural networks (ANN) having many layers where each layer is designed to learn the intricate patterns found in the data. As pointed out earlier, it is nowadays common to build neural networks with thousands of layers, which has been made possible by the advancement in computational power. Because of this fact where ANN tend to use deep layers to learn, the name deep learning was adopted for such algorithms (Zhang et al).

We started by saying that deep learning is a branch of machine learning. The question we might then ask ourselves is, what differentiates the two?

Whilst the performance of classical machine learning algorithms is highly dependent on the correct choice of features and feature engineering techniques applied to the dataset, a task that is no mean feat, in deep learning, this is well taken care of by the layers that have the capability to learn complex relationships of the data. When our dataset is small, the deep learning algorithms are not preferred as they tend to overfit on the dataset and thus provide bad generalization. In such cases, machine learning performs much better than deep learning models.

Having discussed the definition of deep learning and where it is used, let us now learn the building blocks of deep learning algorithms.

1.1 Neurons

A neuron forms the fundamental computational unit of artificial neural networks. Below we examine the functionality of artificial neural networks, let us assume that the predictor variables are x_1 , x_2 , x_3 and x_4 and we are predicting y .

Since the variables will usually not be equally important in predicting the output, we assign weight multipliers to all the inputs to signify importance. Therefore, summing the inputs that have been multiplied by the weights will yield $x_1 \times w_1 + x_2 \times w_2 + x_3 \times w_3 + x_4 \times w_4$. By adding a bias term to the above equation we get $z = (x_1 \times w_1 + x_2 \times w_2 + x_3 \times w_3 + x_4 \times w_4) + b$.

The above equation resembles that of a linear regression. The difference between a neuron and a linear regression is the introduction of a non-linear function called an activation function that is applied to z to predict the output, that is, $y = f(z)$.

```
In [ ]: import matplotlib.pyplot as plt
        from nnv import NNV

        print("\033[1m" + "Fig 1: A Neural Network" + "\033[0m")
        layersList = [
            {"title": "inputs \n + bias", "units": 5, "color": "darkBlue"},
            {"title": "$z$", "units": 1},
            {"title": "$f(z)$", "units": 1, "edges_color": "red", "edges_width": 2},
            {"title": "output", "units": 1, "color": "darkBlue"},
        ]

        NNV(layersList).render(save_to_file="my_example1.png")
        plt.show()
```

So, in summary, given inputs X , a neuron will pair them with weights W and a bias term b , which after adding them will be acted upon by an activation function to yield the output y .

1.2 Neural Network Layers

Nearly all tasks we will face in the industry will need us to stack more layers to learn the intricate relationship present in the data. The organization of layers in a neural network is such that it can allow information to flow from one layer to the next. The layers in a simple neural network are:

- Input layer
- Hidden layer
- Output layer

This can be shown in the diagram below:

```
In [ ]: print("\033[1m" + "Fig 2: Two Layer Neural Network" + "\033[0m")
        layersList = [
            {"title": "input layer", "units": 3, "color": "darkBlue"},
            {"title": "hidden layer", "units": 4},
            {"title": "output layer", "units": 1, "color": "darkBlue"},
        ]

        NNV(layersList).render(save_to_file="my_example2.png")
        plt.show()
```

The input layer consists of the observed variables that are passed to the model. They can be raw and unprocessed data straight from the data source or featured engineered datasets.

The hidden layer is the layer that processes data passed from the input layer to derive complex patterns present in the data. There can be thousands of hidden layers present in a neural network. The processed data is then passed to the output layer.

The output layer gives the output of the modeling exercise. There can be more than one neuron in the output layer depending on the problem being solved.

1.3 Activation Functions

As pointed out earlier, activation functions are used to introduce non-linearity to the input so as to learn the underlying relationship in the data, thus allowing for complex fitting. Without a non-linear transformation, the neural network is a linear regression model.

We will now explore the most commonly used activation functions.

1.3.1 Sigmoid Function (Logistic Function)

The sigmoid function is defined as $f(x) = \frac{1}{1+e^{-x}}$ It gives an S-shaped curve as below:

```
In [ ]: import math

import matplotlib.pyplot as plt
import numpy as np

def sigmoid(x):
    a = []
    for item in x:
        a.append(1 / (1 + math.exp(-item)))
    return a

x = np.arange(-5.0, 5.0, 0.2)
sig = sigmoid(x)

print("\033[1m" + "Fig 3: A sigmoid function curve" + "\033[0m")
plt.plot(x, sig)
# `plt.suptitle`("Fig. 3: A sigmoid function curve")
plt.show()
```

The sigmoid function is differentiable and monotonic and it outputs values between 0 and 1. We can write the sigmoid function in Python as the following:

```
In [ ]: import numpy as np
```

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
```

1.3.2 Tanh Function

The tanh function expressed as $f(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$ resembles the sigmoid function above with the difference being that it outputs values between -1 and 1 as shown below.

```
In [ ]: import matplotlib.pyplot as plt

def tanh(x):
    a = []
    for item in x:
        a.append((1 - math.exp(-2 * item)) / (1 + math.exp(-2 * item)))
    return a

x = np.arange(-5.0, 5.0, 0.2)
tanva = tanh(x)

print("\033[1m" + "Fig 4: A tanh function curve" + "\033[0m")
plt.plot(x, tanva)
plt.show()
```

It can be implemented in Python as shown below:

```
In [ ]: def tanh(x):
    return (1 - np.exp(-2 * x)) / (1 + np.exp(-2 * x))
```

1.3.3 Rectified Linear Unit (ReLU) Function

The ReLU function is mathematically expressed as $f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$ or $f(x) = \max\{0, x\}$ whose outputs are non-negative numbers.

It can pictorially be visualized below.

```
In [ ]: import numpy as np

x = np.linspace(-10, 10, 1000)
y = np.maximum(0, x)

print("\033[1m" + "Fig 5: A ReLU function curve" + "\033[0m")
plt.figure(figsize=(10, 5))
plt.plot(x, y)
plt.legend(["Relu"])
# `plt.suptitle`("Fig. 5: A ReLU function curve")
plt.show()
```

The Python code for ReLU is:

```
In [ ]: def ReLU(x):
    if x < 0:
```

```

        return 0
    else:
        return x

```

1.3.4 The Leakey ReLU Function

The Leakey ReLU is a modification of the ReLU function intended to tackle the dying ReLU problem. It is mathematically expressed as: $f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$ where α usually takes the value 0.01.

It is implemented in Python as:

```

In [ ]: def LeakeyReLU(x, alpha=0.01):
        if x < 0:
            return alpha * x
        else:
            return x

```

Diagrammatically, we can visualize it as:

```

In [ ]: import matplotlib.pyplot as plt
import numpy as np

# Leaky Rectified Linear Unit (Leaky ReLU) Activation Function
def leaky_ReLU(x):
    data = [max(0.05 * value, value) for value in x]
    return np.array(data, dtype=float)

# Generating data For Graph
x_data = np.linspace(-10, 10, 100)
y_data = leaky_ReLU(x_data)

# Graph
print("\033[1m" + "Fig 6: A Leakey ReLU function curve" + "\033[0m")
plt.plot(x_data, y_data)
plt.legend(["leaky_ReLU"])
plt.grid()
plt.show()

```

1.3.5 The Softmax Function

The softmax function generalizes the sigmoid function, and it is used in multi-class classification problems while logistic function is suitable for binary classes. It is expressed as: $f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$ The Python code to produce the above equation is:

```

In [ ]: def softmax(x):
        return np.exp(x) / np.exp(x).sum(axis=0)

```

Other commonly used activation functions are:

a) The exponential linear unit function given by $f(x) = \begin{cases} \alpha (e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$ b) The swish function given by $f(x) = x \sigma(x)$ where $\sigma(x)$ is the sigmoid function.

Now that we have learned the founding blocks of deep learning, let us now apply them in the next section.

2. Forward Propagation

In the previous section, we said that neural networks learn by stacking neurons in layers. A neural network is said to have n -layers if the number of layers excluding the input layer are n . Therefore, a two-layer neural network will have one input layer X , one hidden layer h , and one output layer y as shown below:

```
In [ ]: print("\033[1m" + "Fig 7: A Two Layer Neural Network" + "\033[0m")
layersList = [
    {"title": "input layer", "units": 3, "color": "darkBlue"},
    {"title": "hidden layer", "units": 4},
    {"title": "output layer", "units": 1, "color": "darkBlue"},
]

NNV(layersList).render(save_to_file="my_example3.png")
plt.show()
```

Take a case where our input variables are three, that is, x_1 , x_2 and x_3 as in the diagram above. The number of neurons in the input layer will therefore be three, and in this example, we will also assume the number of neurons in the hidden layer and output layer to be four and one respectively. As earlier indicated, the input variables will be multiplied by their corresponding weights, then we add a bias term before passing this sum to the hidden layer where the activation function will be used.

The obvious question we might be asking ourselves is given the inputs, where do we get the corresponding weights? Practically, it will be impossible to know which of the inputs will be more important in prediction and by what proportion. We therefore assign weights and the bias term using random numbers that we will denote as W_{xh} and b_h respectively, where the dimensions of the weight matrix will have a number of columns equal to the number of neurons in the input layer and a number of rows equal to the number of neurons in the hidden layer so as to ensure matrix multiplication compatibility. Thus, in the presented example, the dimension of W_{xh} is 3×4 and therefore $z_1 = X W_{xh} + b_h$. We pass this equation to the hidden layer where it is acted upon by the activation function. If the activation function in this case is the sigmoid function, then we have $a_1 = \sigma(z_1)$. The outputs a_1 is again multiplied by a new weight matrix and added to a new bias term that is now passed to the output layer. By denoting the weight matrix and bias term here by W_{hy} and b_y respectively, the dimensions of W_{hy} will be 4×1 since the hidden layer has 4 neurons and the output layer has one neuron.

The result is given by $z_2 = a_1 W_{hy} + b_y$

Finally, we apply another activation function to z_2 to get the output value $\hat{y} = \sigma(z_2)$. The above process is what we call the forward propagation.

In summary, the mathematical steps involved in a two-layer neural network is as follows:

$$\begin{cases} z_1 = X W_{xh} + b_h \\ a_1 = \sigma(z_1) \\ z_2 = a_1 W_{hy} + b_y \\ \hat{y} = \sigma(z_2) \end{cases}$$

A Python function implementation of the above steps is as shown below:

```
In [ ]: def forward_prop(X, b_h, b_y, W_x_h, W_h_y):  
        z_1 = np.dot(X, W_x_h) + b_h  
        a_1 = sigmoid(z_1)  
        z_2 = np.dot(a_1, W_h_y) + b_y  
        y_hat = sigmoid(z_2)  
  
        return y_hat
```

Are we done? Apparently not. We have to compare our approximated output with the actual output. How do we therefore know that our model is performing up to expectation? We define a cost/loss function. Just as there are many activation functions, there exist many cost functions. To build the intuition in the next section, we will use the cost function below $J = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y})^2$ the sum of the squared difference between the actual output and the predicted output where n is the number of variables used in training.

Now that we have learned how to predict the output of a neural network, in the next section, we are going to fine-tune this to get better approximations.

3. Backpropagation

At this time, the question that is lingering in our heads might be how does a neural network learn to predict the correct output? Where is the cost function coming in in all of this? A short answer to this is that we aim to have the cost function as small as possible, and therefore for any iteration we will encounter, we will seek to minimize the cost function. But why? If the cost function is large, then it means that the difference between the predicted and actual outputs is high.

The only way we can minimize the cost function is to change some parameters, but we cannot change the input and output values. Therefore, we can only change the weight and bias terms since they were initially assigned random numbers. The role of backpropagation will be to help the neural network learn how to update the weights in such a manner so as to minimize the cost function thereby increasing the accuracy of our prediction. We use a method called gradient descent to update the weights.

Gradient descent is an optimization algorithm commonly used in machine learning to find the optimal values of weights that will minimize the cost function or errors.

To understand how gradient descent finds the optimal weights, let us provide an intuitive explanation. From its name "Gradient descent," think of going down a slope. The

technique tries to find a combination of parameters in a curve that will ensure the output gives a low-cost value.

Since our interest is to find the optimal weight, consider the curve below as a function of w , that is, $E(w)$ as shown below:

```
In [ ]: import matplotlib.pyplot as plt
import numpy as np

w = np.linspace(-3, 3, num=100)

Ew = []
for i in range(len(w)):
    Ew.append(w[i]**4 - 5 * w[i]**2 - 3 * w[i])

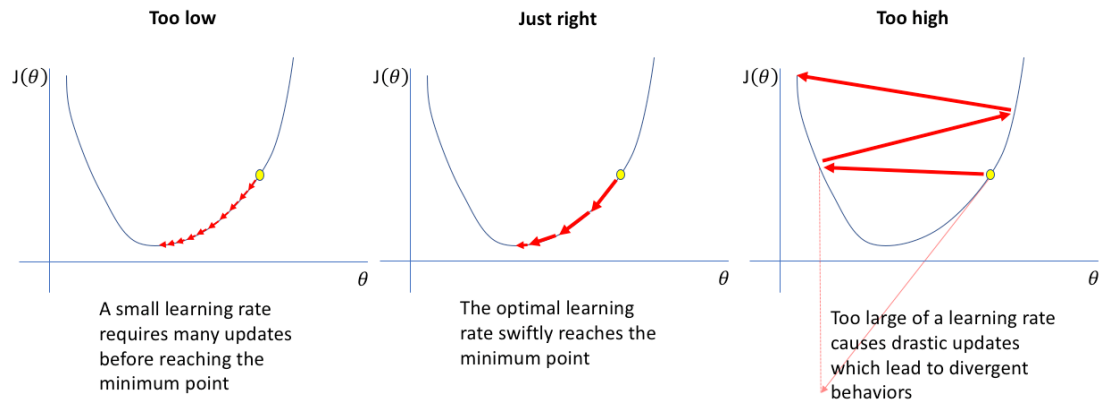
print("\033[1m" + "Fig 8: A Non-convex Curve" + "\033[0m")
plt.plot(w, Ew)
plt.grid()
plt.axvline()
plt.axhline()
plt.show()
```

Our ideal will be to hit the global minimum, but gradient (the lowest trough in the curve) descent will not necessarily get to this point, though it will get close. We start first by selecting initial values of w . The gradient descent will attempt to reach the local minimum point, and since it will only descend and not climb, by increasing or decreasing the value of w gradually in small steps, we will see it go down the slope toward the global minimum. We refer to these steps as the learning rate denoted by α .

By repeating this process over many iterations, the gradient descent will give us optimal weights that ensure the cost is at a minimum. Remember that by forward propagation we stopped at the output layer. By back-propagation, we will move in the reverse order, that is, from the output layer to the input layer while calculating the cost function's gradient that minimizes the error. Having calculated the gradients, we can now proceed to update the weights as follows: $W_{\text{new}} = W - \alpha \frac{\partial J}{\partial W}$ where α is the learning rate.

The learning rate guides the type of step to be taken, that is, a small learning rate infers we take a small step down the slope and therefore the gradient descent process becomes slow. The inverse is also true as shown in the diagrams below.

Fig. 9: Gradient Descent with Different Learning Rates



Source: [Jeremy Jordan, Setting the Learning Rate of Your Neural Network, 2018](#)

This whole procedure of moving from the output layer to the input layer while updating the weights using an optimization algorithm is called back-propagation.

In the two-layer neural network example we considered in the last section, we had two weights, W_{hx} that feeds inputs to hidden layer and W_{hy} that feeds to the output layer from the hidden layer. To find the optimal weights that gives us minimal errors, we will find the derivative of J with respect to the weights.

We will start with W_{hy} since we are moving backwards, from the output layer towards the input layer. Recall that cost function J is given by: $J = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y})^2$ and $\hat{y} = \sigma(z_2)$ where $z_2 = a_1 W_{\text{hy}} + b_y$. To find the derivative of J with respect to W_{hy} we will use the chain rule, that is,

$$\frac{\partial J}{\partial W_{\text{hy}}} = \frac{\partial J}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z_2} \times \frac{\partial z_2}{\partial W_{\text{hy}}} \quad (1)$$

Note that: $\frac{\partial J}{\partial \hat{y}} = (y - \hat{y})$ and $\frac{\partial \hat{y}}{\partial z_2} = \sigma'(z_2)$ where σ' denotes the derivative of the sigmoid activation function.

The derivative of the sigmoid function $\sigma(z) = \frac{1}{1 + e^{-z}}$ is $\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2}$. Finally, $\frac{\partial z_2}{\partial W_{\text{hy}}} = a_1$. Combining the derivatives back to the equation (1) above, we get

$$\frac{\partial J}{\partial W_{\text{hy}}} = (y - \hat{y}) \times \sigma'(z_2) \times a_1 \quad (2)$$

Next, we now find the derivative of J with respect to the weight W_{hx} . Recall that: $\hat{y} = \sigma(z_2)$, $z_2 = a_1 W_{\text{hy}} + b_y$, $a_1 = \sigma(z_1)$, $z_1 = X W_{\text{hx}} + b_x$. By the chain rule, we get

$$\frac{\partial J}{\partial W_{\text{hx}}} = \frac{\partial J}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z_2} \times \frac{\partial z_2}{\partial a_1} \times \frac{\partial a_1}{\partial z_1} \times \frac{\partial z_1}{\partial W_{\text{hx}}} \quad (3)$$

where: $\frac{\partial z_2}{\partial a_1} = W_{\text{hy}}$ and $\frac{\partial a_1}{\partial z_1} = \sigma'(z_1)$. So

$$\frac{\partial J}{\partial W_{\text{hx}}} = (y - \hat{y}) \times \sigma'(z_2) \times W_{\text{hy}} \times \sigma'(z_1) \times X \quad (4)$$

Therefore: $\frac{\partial J}{\partial W_{\text{hx}}} = (y - \hat{y}) \times \sigma'(z_2) \times W_{\text{hy}} \times \sigma'(z_1) \times X$

Since we now have the gradients, we can now update the weights as: $W_{hy} = W_{hy} - \alpha \frac{\partial J}{\partial W_{hy}}$ (5) $W_{xh} = W_{xh} - \alpha \frac{\partial J}{\partial W_{xh}}$ (6)

Let us now implement backpropagation in Python. We will define dW_{hy} and dW_{xh} to represent $\frac{\partial J}{\partial W_{hy}}$ and $\frac{\partial J}{\partial W_{xh}}$ respectively.

We first write a function for the derivative of the sigmoid function.

```
In [ ]: def sigmoid_derivative(z):
        return np.exp(-z) / ((1 + np.exp(-z)) ** 2)
```

Define $(y - \hat{y}) \sigma'(z_2)$ as:

```
delta2 = np.multiply((y_hat - y), sigmoid_derivative(z_2))
```

The gradient with respect to W_{hy} is therefore:

```
dWhy = np.dot(a1.T, delta2)
```

and with respect to W_{xh} is:

```
delta1 = np.dot(delta2, Why.T) * sigmoid_derivative(z1)
```

```
dWxh = np.dot(X.T, delta1)
```

We therefore update the weights accordingly:

```
Wxh = Wxh - alpha * dWxh
```

```
Why = Why - alpha * dWhy
```

So, the function becomes:

```
In [ ]: def backward_prop(y_hat, z_1, a_1, z_2, alpha, W_x_h, W_h_y, X):
        delta2 = np.multiply((y_hat - y), sigmoid_derivative(z_2))
        d_W_h_y = np.dot(a_1.T, delta2)

        delta1 = np.dot(delta2, W_h_y.T) * sigmoid_derivative(z_1)
        d_W_x_h = np.dot(X.T, delta1)

        W_x_h = W_x_h - alpha * d_W_x_h
        W_h_y = W_h_y - alpha * d_W_h_y

        return W_x_h, W_h_y
```

4. Conclusion

In this lesson, we started by introducing the concept of deep learning and its application areas in financial engineering. We started by discussing the fundamental building blocks of deep learning, after which we discussed how neural networks get the output

predictions using forward prediction. Then, we fine-tuned the algorithm by introducing the concept of gradient descent in back-propagation.

In the next lesson, we will put these concepts together as we work on examples and compare the performance of deep learning and other algorithms.

References

1. Addo, Martey Peter et al. "Credit Risk Analysis Using Machine and Deep Learning Models." *Risks*, vol. 6, no. 2, 2018, pp. 1-20.
2. Buehler, Hans et al. "Deep Hedging." *Quantitative Finance*, vol. 19, no. 8. 2019, pp. 1271-1291.
3. Maeda, Iwao et al. "Deep Reinforcement Learning in Agent Based Financial Market Simulation." *Journal of Risk and Financial Management*, vol. 13, no. 4, 2020.
4. Mai, Feng, et al. "Deep Learning Models for Bankruptcy Prediction Using Textual Disclosures." *European Journal of Operational Research*, vol. 274, no. 2, 2019, pp. 743-758.
5. Modarres, Ceena et al. "Towards Explainable Deep Learning for Credit Lending: A Case Study." *arXiv preprint*, arXiv:1811.06471, 2018.
6. Nixon, Jeremy et al. "Measuring Calibration in Deep Learning." *CVPR Workshops*, vol. 2, no. 7, 2019.
7. Sezer, Omer Berat et al. "Financial Time Series Forecasting with Deep Learning: A Systematic Literature Review: 2005-2019." *Applied Soft Computing*, vol. 90, 2020, 1-63.
8. Yao, Jingtao et al. "Option Price Forecasting Using Neural Networks." *Omega*, vol. 28, no. 4, 2000, pp. 455-466.
9. Qu, Yi et al. "Review of Bankruptcy Prediction Using Machine Learning and Deep Learning Techniques." *Procedia Computer Science*, vol. 162, 2019, pp. 895-899.
10. Zhang, Aston et al. "Dive into Deep Learning." *arXiv preprint* arXiv:2106.11342, 2021.
<https://d2l.ai/d2l-en.pdf>